

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What is the output?

```
def fun(x):
    if x == 0:
        return
    return x + fun(x - 1)
```

```
print(fun(3))
```

- A)3
- B)None
- C)5
- D)6

Q2. What will normally happen?

```
import math
import math
```

- A)The module is normally loaded and executed twice
- B)ImportError
- C)SyntaxError
- D)The module is normally loaded once and reused

Q3. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

- A)Nothing
- B)A and B
- C)A
- D)B

Q4. What is the output?

```
def outer():
    x = 10
```

```
def inner():
    return x + 5

return inner
```

```
f = outer()
```

```
print(f())
```

- A)10
- B)NameError
- C)None
- D)15

Q5. What will happen?

```
def fun(n):
    if n == 0:
        return 1
    return n * fun(n)
```

```
print(fun(3))
```

- A)TypeError
- B)RecursionError
- C)3
- D)6

Q6. What is the output?

```
def fun(a=10, b=20):
    return a - b
```

```
print(fun(30))
```

- A)10
- B)30
- C)-10
- D)20

Q7. What happens when this code executes?

```
def fun():
    print(variable)
    variable = 50
```

```
variable = 10
```



fun()

A)50

B)10

C)NameError

D)UnboundLocalError

Q8. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

A)15

B)5

C>Error

D)10

Q9. What is the output?

```
try:
```

```
    x = int("abc")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)A C

B)B

C)B C

D)A

Q10. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

A)NameError

B)6

C)36

D)6.0

Q11. What is the output?

```
def fun(x):
```

```
    x += 10
```

```
    return x
```

```
x = 5
```

```
y = fun(x)
```

```
print(x, y)
```

A)5 15

B)15 5

C)15 15

D)5 5

Q12. What happens here?

```
try:
```

```
    raise ValueError("Wrong value")
```

```
except ValueError as e:
```

```
    print(e)
```

A)ValueError

B)e

C)Nothing

D)Wrong Value

Q13. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(
```

```
    lambda a, b: a * b,
```

```
    numbers
```

```
)
```

```
print(result)
```

A)9

B)24

C)0

D)10

Q14. What will be printed?

```
class InvalidAgeError(Exception):
```

```
    pass
```

```
def check_age(age):
```

```
    if age < 18:
```

```
        raise InvalidAgeError("Age is too low")
```

```
    return "Allowed"
```

```
try:
```

```
    print(check_age(15))
```

```
except InvalidAgeError as e:
```

```
    print(type(e).__name__)
```

```
    print(e)
```



A)Exception
Age is too low
B)Nothing, because custom exceptions cannot be caught
C)InvalidAgeError
Age is too low
D)InvalidAgeError
15

Q15. What will be the output?

```
x = 10
```

```
def outer():  
    x = 20
```

```
    def inner():  
        global x  
        x = 30
```

```
    inner()  
    print(x)
```

```
outer()  
print(x)
```

A)30
20
B)20
10
C)20
30
D)30
30

Q16. What is the output?

```
try:  
    print(10 / 0)  
except Exception:  
    print("A")  
except ZeroDivisionError:  
    print("B")
```

A)A
B)A B
C)SyntaxError
D)B

Q17. Which object keeps track of modules that have already been imported during the current Python process?

A)sys.path
B).__module__
C)sys.imports
D)sys.modules

Q18. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

A)sqrt(25)
B)25
C)5
D)5.0

Q19. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went wrong")
```

```
try:  
    test()  
except MyError as e:  
    print(type(e).__name__)
```

A)Something went wrong
B)Exception
C)MyError
D)TypeError

Q20. What will be printed?

```
def make_counter():  
    count = 0
```

```
    def counter():  
        nonlocal count  
        count += 1  
        return count
```

```
    return counter
```

```
a = make_counter()  
b = make_counter()
```

```
print(a())  
print(a())  
print(b())
```



print(a())

A)1

2

3

4

B)0

1

0

2

C)1

2

1

3

D)1

1

1

1

Q21. What is the output?

try:

raise ValueError

except ValueError:

print("A")

raise

A)A, and execution ends normally

B)B

C)A, followed by propagation of the same exception

D)No Output

Q22. What happens when this function is called?

def fun(a=10, b):

return a + b

A)It executes correctly

B)NameError

C)SyntaxError

D)TypeError

Q23. What does the following function return?

def fun():

print("Hello")

x = fun()

print(x)

A>Error

B)Hello None

C)Hello

D)None

Q24. Suppose module_a.py contains:

print("Module loaded")

value = 10

And another file contains:

import module_a

import module_a

print(module_a.value)

Assuming both imports occur in the same Python process, what is normally printed?

A)Module loaded

Module loaded

10

B)ImportError

C)Module loaded

10

D)10

10

Q25. What is the purpose of this statement?

if __name__ == "__main__":

A)It prevents all imports

B)It identifies code intended to run when the file is executed directly

C)It creates the main function

D)It prevents syntax errors

Q26. What is the output?

def fun(x):

return x * 2

a = fun

print(a(5))

A)func(5)

B>Error

C)10

D)5



Q27. What will be the output?

```
def fun(x):
    try:
        if x < 0:
            raise ValueError("negative")
        return x
    except ValueError:
        return 100
    finally:
        print("cleanup")
```

print(fun(-5))

- A)negative
- cleanup
- B)cleanup
- 100
- C)cleanup
- 5
- D)100
- cleanup

Q28. What will be printed?

x = 10

f = lambda: x

x = 20

print(f())

- A)None
- B)NameError
- C)20
- D)10

Q29. What will be printed?

```
def fun():
    print("A")
    return
    print("B")
```

fun()

- A)A B
- B)A
- C)Nothing
- D)B

**Q30. if test.py has
print("A")**

```
if __name__ == "__main__":
    print("B")
```

If test.py is executed directly, what will be printed?

- A)A followed by B
- B)B
- C)A
- D)Nothing

Q31. What happens here?

```
def fun(a):
    return a
```

print(fun())

- A)NameError
- B)TypeError
- C)0
- D)None

Q32. What will be printed?

```
try:
    print(10 / 0)
except ZeroDivisionError:
    print("A")
except Exception:
    print("B")
```

- A)Nothing
- B)B
- C)A
- D)A B

Q33. What is the major difference between the following?

import math

and

from math import sqrt

- A)There is no difference
- B)First imports only sqrt
- C)First provides the module namespace; second imports sqrt directly
- D)Both import only sqrt

Q34. What will be printed?

```
try:
    x = int("10")
```



except ValueError:

print("A")

else:

print("B")

finally:

print("C")

A)A

B)B C

C)B

D)A C

Q35. What will be printed?

def fun(a, b):

return a + b

print(fun(10, b=20))

A)TypeError

B)SyntaxError

C)30

D)20

Q36. What happens when this code runs?

def first():

second()

def second():

third()

def third():

raise RuntimeError("Failure")

try:

first()

except RuntimeError:

print("Handled")

A)Handled

B)RuntimeError cannot propagate through functions

C)NameError

D)TypeError

Q37. What is the output?

def fun():

try:

return 10

finally:

return 20

print(fun())

A)RuntimeError

B)10

C)None

D)20

Q38. What is the output?

def fun():

try:

print("A")

raise ValueError

except ValueError:

print("B")

return 10

else:

print("C")

return 20

finally:

print("D")

print(fun())

A)A

B

10

D

B)A

B

C

D

10

C)A

B

D

10

D)A

C

D

20

Q39. What will be printed?

def fun(x=[]):

x.append(10)

return x

a = fun()

b = fun()



```
print(a)
```

```
print(b)
```

A)Error

B)[10]

[10]

C)[10, 10]

[10]

D)[10]

[10, 10]

Q40. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal x
```

```
        x = x + 5
```

```
        return x
```

```
    inner()
```

```
    return x
```

```
print(outer())
```

A)UnboundLocalError

B)10

C)5

D)15

Q41. Suppose sample.py contains:

```
x = 10
```

```
def display():
```

```
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)
```

```
sample.display()
```

A)x

display

B>Error

C)10

Hello

D)Only Hello

Q42. What will be the output?

```
x = 10
```

```
def fun():
```

```
    print(x)
```

```
    x = 20
```

```
fun()
```

A)20

B)NameError

C)10

D)UnboundLocalError

Q43. What happens?

```
def fun(a, b):
```

```
    return a + b
```

```
print(fun(10, a=20))
```

A)30

B)20

C)TypeError

D)SyntaxError

Q44. What will happen?

```
def fun(a, b=10):
```

```
    return a + b
```

```
print(fun(b=5, a=20))
```

A)TypeError

B)SyntaxError

C)25

D)15

Q45. What will be printed?

```
def fun():
```

```
    try:
```

```
        return 10
```

```
    finally:
```

```
        print("Finally")
```

```
print(fun())
```

A)Finally only

B)10 only

C)10 followed by Finally

D)Finally followed by 10

Q46. What is the output?



```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
numbers)
    )
)
```

```
print(result)
```

A)[2, 4, 6, 8, 10]

B)[4, 8]

C)[1, 3, 5]

D)[2, 4]

Q47. Why is

try:

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

codes's second handler never reached?

A)except blocks execute randomly

B)ZeroDivisionError occurs before Exception

C)Exception is a superclass of ZeroDivisionError

D)ZeroDivisionError is not an exception

Q48. What will be printed?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        print(x)
```

```
    x = 30
```

```
    inner()
```

```
outer()
```

A)10

B)NameError

C)20

D)30

Q49. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

A)random cannot be imported

B)Python will always reject it

C)The local file can shadow the intended standard-library module

D)Python automatically renames the local file

Q50. What will happen?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal y
```

```
        y = 20
```

```
    inner()
```

```
outer()
```

A)NameError

B)10

C)20

D)SyntaxError

